

VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Safe Transfer: Lost-Update and Deadlock Prevention in Banking

A9512 – DATA BASE MANAGEMENT SYSTEM LABORATORY

Submitted by

J. Sai Deekshith 25881A0546
T. Sai Purnanand 25881A0549
G. Yashwanth 25881A0562

Course Instructor

Ms. Shaik Shahanaz
Assistant Professor
CSE department

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH**,
Approved by **AICTE**, Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad - 501 218,
Telangana, India.

Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Course End Project titled **"Safe Transfer: Lost-Update and Deadlock Prevention in Banking"** is carried out by **"Jammullayyala Sai Deekshith"**, **"Toranagatti Sai Purnanand"**, and **"Gangarapu Yashwanth"** with Roll Numbers **"25881A0546"**, **"25881A0549"**, and **"25881A0562"** respectively towards **A9512 – Database Management Systems Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic Year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Shaik Shahanaz	**Dr. R. Karthikeyan**	**Prof. Y. Vijayalata**
Assistant Professor, CSE	Professor, CSE	Department of CSE

Contents

Abstract **3**

1 Introduction **4**

2 Problem Statement **4**

3 Objectives **5**

4 Methodology **5**
4.1 Block Diagram 5
4.2 ER Diagram and Relational Schema 6
4.3 Tools Used 6
4.4 How ACID Is Achieved 7
4.5 Transaction States 7

5 Implementation **7**
5.1 Database Creation Script 7
5.2 Stored Procedures 9

6 Schedules and Serializability **12**

7 Execution and Results **13**
7.1 Test 1: Successful Transfer (Atomicity and Durability) 13
7.2 Test 2: Deposit, Withdrawal and Mini Statement (Cursor) 14
7.3 Test 3: Failed Transfer Is Rolled Back (Atomicity) 14
7.4 Test 4: Integrity Constraint (Consistency) 15
7.5 Test 5: Lost Update Without Locking (Isolation Problem) 16
7.6 Test 6: Lost Update Fixed With Row Locking 16
7.7 Test 7: Deadlock (ORA-00060) 17
7.8 Test 8: Deadlock Prevention by Ordered Locking 19
7.9 Test 9: Lock Wait Control (NOWAIT) and Serializable Isolation 19

8 Results and Discussion **20**

9 Mapping POs and SDGs **21**
9.1 Program Outcomes (POs) Mapping 21
9.2 Sustainable Development Goals (SDGs) Mapping 21

10 Conclusion **22**

11 Future Scope **22**

Bibliography **23**

Abstract

Banking applications handle money, so every update must stay correct even when a system fails midway or many customers use the same accounts at the same time. A fund transfer changes several records together, and if only some of the steps are completed, money can be lost or created. When two sessions read and write the same account without control, one update can overwrite the other, and two transfers in opposite directions can block each other forever. These problems are hard to see in normal use, so they must be reproduced and studied on purpose.

This project presents SafeTransfer, a small banking database built on Oracle 10g that demonstrates how transaction management and concurrency control keep data correct. The database has four tables: customers, accounts, transactions and an audit log. All money operations (deposit, withdrawal and transfer) are written as PL/SQL stored procedures. Each procedure commits when every step succeeds and rolls back when any step fails, which gives atomicity and durability. Integrity constraints such as primary keys, foreign keys and CHECK rules reject invalid data like a negative balance, which protects consistency. A trigger records the old and new balance of every change in the audit table within the same transaction, so every change can be traced.

The project then tests concurrency using two SQL*Plus sessions. A lost update is reproduced first, where a withdrawal of 500 is overwritten and the final balance is 9,000 instead of 8,500. It is fixed by locking the row with `SELECT ... FOR UPDATE`, which makes the second session wait and gives the correct result. A deadlock is then reproduced with two transfers moving money in opposite directions, and Oracle detects it with error `ORA-00060`. Deadlock is finally prevented by always locking the account with the lower number first, which removes the circular wait. The experiments are connected to the course theory of schedules, conflict serializability, recoverability and strict two-phase locking.

The results show that stored procedures, constraints, triggers and lock-based protocols together keep balances correct, and that preventing a deadlock is better than detecting it and retrying. The project turns the ACID properties and the lock-based concurrency control from Unit-IV and Unit-V into working, repeatable examples.

Keywords: DBMS, Oracle 10g, PL/SQL, stored procedures, ACID properties, transaction management, concurrency control, lost update, deadlock prevention, row-level locking, serializability, audit trigger

1 Introduction

Banks and payment systems deal with money, so every operation they perform must be exact. A customer who transfers funds expects the amount to leave one account and arrive in the other, once and only once. Behind this simple action, a database must update several records together, and it must do so correctly even when a failure occurs midway. Transaction management in a Database Management System exists to give this assurance.

The difficulty grows when many users work at the same time. If two operations read the same balance and then write back their own results, one of them can silently overwrite the other, which is called a lost update. If two transfers move money in opposite directions and each waits for an account the other has locked, neither can continue, which is a deadlock. Problems like these can leave wrong balances or stop the system, and they cannot be seen unless they are deliberately reproduced.

BankSafe is a small banking database built on Oracle 10g to study these issues. It stores customers, accounts, transactions and an audit log, and all money operations (deposit, withdrawal and transfer) are written as PL/SQL stored procedures that either commit completely or roll back. Integrity constraints reject invalid data such as a negative balance, and a trigger records every change of balance in the audit table, so the full history can be checked later.

The project then tests these ideas in practice. Using two SQL*Plus sessions, it reproduces a lost update and a deadlock, and fixes them with row locking (`SELECT ... FOR UPDATE`) and a fixed order of locking accounts. In this way, BankSafe shows the ACID properties and the lock-based concurrency control taught in Unit-IV and Unit-V of the course through working examples. The rest of this report explains the design, the implementation, the execution results and the conclusions.

2 Problem Statement

In a banking application, a money transfer changes more than one record: it reduces the balance of one account, increases the balance of another, and records the event. These steps must succeed or fail together. They must also stay correct when many users act on the same accounts at the same time. With simple SQL and a read-then-write coding pattern, two things can go wrong. Lost update: two sessions read the same balance, and the second one to write overwrites the result of the first. Deadlock: two transfers in opposite directions each hold one account and wait for the other, so neither can finish. The problem is to design and implement a banking database on Oracle 10g that keeps balances correct under failures and concurrent access. The system must prevent the lost update and prevent deadlock, instead of only detecting it after the damage is done.

3 Objectives

- To design a normalised relational schema for accounts, customers, transactions and an audit trail.
- To implement deposits, withdrawals and transfers as PL/SQL stored procedures with commit and rollback.
- To demonstrate the four ACID properties with working examples.
- To reproduce a lost update and a deadlock using two concurrent sessions.
- To resolve both problems using lock-based protocols and a deadlock-prevention rule.
- To relate the experiments to schedules, serializability and recoverability.

4 Methodology

4.1 Block Diagram

Figure 1 shows how a client session reaches the data. Every operation goes through a stored procedure, which uses the Oracle lock manager and redo log to keep the data consistent and durable.

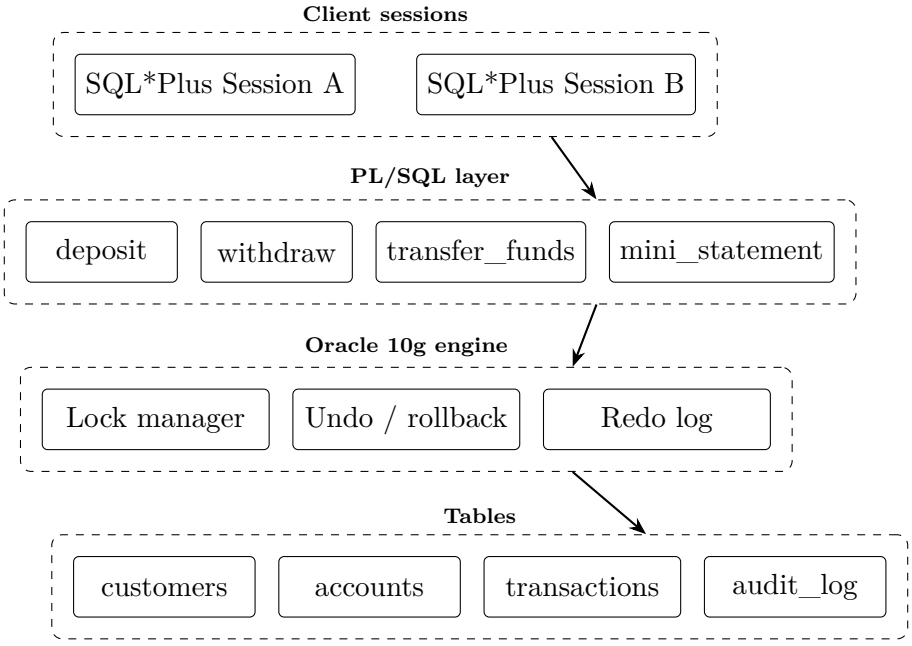


Figure 1: Architecture of BankSafe

4.2 ER Diagram and Relational Schema

A customer owns one or more accounts; an account has many transactions and many audit records. Figure 2 shows the ER diagram and Table 1 lists the relational schema obtained by the E-R to relational mapping. All tables are in 3NF and BCNF, since every non-key attribute depends only on the key of its own table.

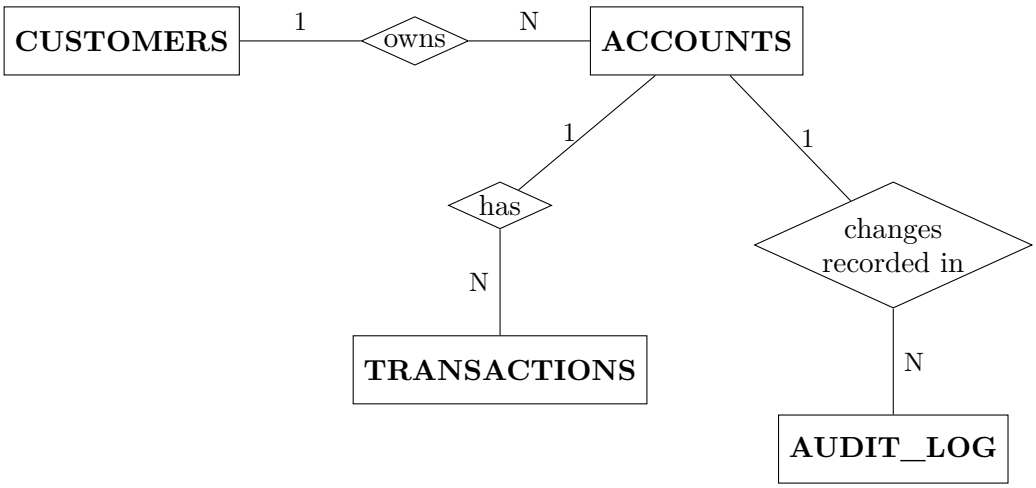


Figure 2: ER diagram of BankSafe

Table 1: Relational schema (primary keys underlined in text, foreign keys marked FK)

Table	**Attributes**
customers	cust_id, cust_name, phone (unique), city
accounts	acc_no, cust_id (FK), acc_type, balance (CHECK ≥ 0), opened_on
transactions	txn_id, acc_no (FK), txn_type, amount (CHECK > 0), txn_time, related_acc
audit_log	log_id, acc_no, old_balance, new_balance, changed_by, changed_on

4.3 Tools Used

- **Database:** Oracle Database 10g, SQL*Plus Release 10.2.0.1.0 (two sessions open side by side).
- **Language:** SQL and PL/SQL (procedures, triggers, cursors, exceptions).
- **Scripts:** `01_setup.sql` (schema, data, trigger) and `02_procedures.sql` (stored procedures).

4.4 How ACID Is Achieved

ACID is the set of four guarantees that a database transaction must give. **Atomicity** means all steps of a transaction happen or none of them do. **Consistency** means a transaction moves the database from one valid state to another, so every rule (constraint) still holds afterwards. **Isolation** means transactions running at the same time do not disturb each other's work. **Durability** means that once a transaction is committed its changes survive a crash. Table 2 shows which BankSafe feature provides each property.

Table 2: Mapping of ACID properties to BankSafe features

Property	**Mechanism in BankSafe**
Atomicity	Debit, credit, and transaction records are saved together by one `COMMIT`; any error runs `ROLLBACK` so nothing is saved.
Consistency	Primary/foreign keys and `CHECK` constraints (balance ≥ 0, amount > 0) keep the database valid before and after each transaction.
Isolation	Row locks through `SELECT ... FOR UPDATE` stop two sessions from changing the same account at once; serializable mode is also tested.
Durability	After `COMMIT`, Oracle has written the change to the redo log, so it survives a crash.

4.5 Transaction States

Each procedure call moves through the standard states: *active* (statements executing), *partially committed* (last statement done), *committed* (after `COMMIT`), or *failed* followed by *aborted* (after `ROLLBACK` when an exception such as insufficient funds occurs).

5 Implementation

5.1 Database Creation Script

The script below creates the tables with their integrity constraints, two sequences (Oracle 10g has no identity columns), the audit trigger and the sample data (accounts 101, 102, 103 with balances 10,000, 5,000 and 20,000).

Listing 1: `01_setup.sql`

```
-- BankSafe : table, sequence, sample data and trigger script (Oracle 10g)
SET ECHO ON
SET LINESIZE 100
SET PAGESIZE 40

-- clean up (ignore errors on the very first run)
DROP TABLE audit_log;
DROP TABLE transactions;
DROP TABLE accounts;
```

```
DROP TABLE customers;
DROP SEQUENCE txn_seq;
DROP SEQUENCE audit_seq;

CREATE TABLE customers (
  cust_id    NUMBER(6)    PRIMARY KEY,
  cust_name  VARCHAR2(50) NOT NULL,
  phone      VARCHAR2(15) UNIQUE,
  city       VARCHAR2(30)
);

CREATE TABLE accounts (
  acc_no     NUMBER(8)     PRIMARY KEY,
  cust_id    NUMBER(6)     NOT NULL REFERENCES customers(cust_id),
  acc_type   VARCHAR2(10)  DEFAULT 'SAVINGS' NOT NULL,
  balance    NUMBER(12,2)  DEFAULT 0 NOT NULL,
  opened_on  DATE          DEFAULT SYSDATE,
  CONSTRAINT chk_balance  CHECK (balance >= 0),
  CONSTRAINT chk_acc_type CHECK (acc_type IN ('SAVINGS','CURRENT'))
);

CREATE SEQUENCE txn_seq   START WITH 1001 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE audit_seq START WITH 1    INCREMENT BY 1 NOCACHE;

CREATE TABLE transactions (
  txn_id       NUMBER(10)    PRIMARY KEY,
  acc_no       NUMBER(8)     NOT NULL REFERENCES accounts(acc_no),
  txn_type     VARCHAR2(15)  NOT NULL,
  amount       NUMBER(12,2)  NOT NULL,
  txn_time     DATE          DEFAULT SYSDATE,
  related_acc  NUMBER(8),
  CONSTRAINT chk_txn_type CHECK (txn_type IN ('DEPOSIT','WITHDRAW','TRANSFER_OUT','
    TRANSFER_IN')),
  CONSTRAINT chk_amount   CHECK (amount > 0)
);

CREATE TABLE audit_log (
  log_id       NUMBER(10)    PRIMARY KEY,
  acc_no       NUMBER(8)     NOT NULL,
  old_balance  NUMBER(12,2),
  new_balance  NUMBER(12,2),
  changed_by   VARCHAR2(30),
  changed_on   DATE
);

-- every balance change is written to audit_log inside the SAME transaction
CREATE OR REPLACE TRIGGER trg_audit_balance
AFTER UPDATE OF balance ON accounts
FOR EACH ROW
BEGIN
```

```
  INSERT INTO audit_log (log_id, acc_no, old_balance, new_balance, changed_by,
    changed_on)
  VALUES (audit_seq.NEXTVAL, :OLD.acc_no, :OLD.balance, :NEW.balance, USER, SYSDATE);
END;
/

INSERT INTO customers VALUES (1, 'Ravi Kumar',  '9000000001', 'Hyderabad');
INSERT INTO customers VALUES (2, 'Anita Rao',   '9000000002', 'Warangal');
INSERT INTO customers VALUES (3, 'Suresh Babu', '9000000003', 'Vijayawada');

INSERT INTO accounts (acc_no, cust_id, acc_type, balance) VALUES (101, 1, 'SAVINGS',
    10000);
INSERT INTO accounts (acc_no, cust_id, acc_type, balance) VALUES (102, 2, 'SAVINGS',
    5000);
INSERT INTO accounts (acc_no, cust_id, acc_type, balance) VALUES (103, 3, 'CURRENT',
    20000);
COMMIT;

COLUMN balance     FORMAT 999,990.00
COLUMN amount      FORMAT 999,990.00
COLUMN old_balance FORMAT 999,990.00
COLUMN new_balance FORMAT 999,990.00
COLUMN txn_type    FORMAT A12
COLUMN changed_by  FORMAT A10
```

5.2 Stored Procedures

`transfer_funds` is the core of the project. It locks both accounts in ascending account-number order (deadlock prevention), checks the balance, performs the debit and credit, writes two transaction records and commits. Any exception rolls the whole transfer back.

Listing 2: `02_procedures.sql`

```
-- BankSafe : stored procedures (Oracle 10g)
SET SERVEROUTPUT ON

CREATE OR REPLACE PROCEDURE deposit (p_acc IN NUMBER, p_amt IN NUMBER) IS
BEGIN
  IF p_amt <= 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'Amount must be positive');
  END IF;
  UPDATE accounts SET balance = balance + p_amt WHERE acc_no = p_acc;
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20003, 'Account not found');
  END IF;
  INSERT INTO transactions (txn_id, acc_no, txn_type, amount)
  VALUES (txn_seq.NEXTVAL, p_acc, 'DEPOSIT', p_amt);
  COMMIT;                -- Durability
EXCEPTION
  WHEN OTHERS THEN
```

```
    ROLLBACK;            -- Atomicity
    RAISE;
END deposit;
/

CREATE OR REPLACE PROCEDURE withdraw (p_acc IN NUMBER, p_amt IN NUMBER) IS
  v_bal accounts.balance%TYPE;
BEGIN
  IF p_amt <= 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'Amount must be positive');
  END IF;
  -- lock the row first, then check the balance (no lost update possible)
  SELECT balance INTO v_bal FROM accounts WHERE acc_no = p_acc FOR UPDATE;
  IF v_bal < p_amt THEN
    RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds in account ' || p_acc);
  END IF;
  UPDATE accounts SET balance = balance - p_amt WHERE acc_no = p_acc;
  INSERT INTO transactions (txn_id, acc_no, txn_type, amount)
  VALUES (txn_seq.NEXTVAL, p_acc, 'WITHDRAW', p_amt);
  COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20003, 'Account not found');
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END withdraw;
/

CREATE OR REPLACE PROCEDURE transfer_funds (
  p_from IN NUMBER,
  p_to   IN NUMBER,
  p_amt  IN NUMBER
) IS
  v_first  accounts.acc_no%TYPE;
  v_second accounts.acc_no%TYPE;
  v_dummy  accounts.balance%TYPE;
  v_bal    accounts.balance%TYPE;
BEGIN
  IF p_amt <= 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'Amount must be positive');
  END IF;
  IF p_from = p_to THEN
    RAISE_APPLICATION_ERROR(-20004, 'Source and destination are the same');
  END IF;

  -- Deadlock prevention: always lock the lower account number first
  IF p_from < p_to THEN
    v_first := p_from;  v_second := p_to;
```

```
  ELSE
    v_first := p_to;    v_second := p_from;
  END IF;
  SELECT balance INTO v_dummy FROM accounts WHERE acc_no = v_first  FOR UPDATE;
  SELECT balance INTO v_dummy FROM accounts WHERE acc_no = v_second FOR UPDATE;

  SELECT balance INTO v_bal FROM accounts WHERE acc_no = p_from;
  IF v_bal < p_amt THEN
    RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds in account ' || p_from);
  END IF;

  UPDATE accounts SET balance = balance - p_amt WHERE acc_no = p_from;
  UPDATE accounts SET balance = balance + p_amt WHERE acc_no = p_to;

  INSERT INTO transactions (txn_id, acc_no, txn_type, amount, related_acc)
  VALUES (txn_seq.NEXTVAL, p_from, 'TRANSFER_OUT', p_amt, p_to);
  INSERT INTO transactions (txn_id, acc_no, txn_type, amount, related_acc)
  VALUES (txn_seq.NEXTVAL, p_to, 'TRANSFER_IN', p_amt, p_from);
  COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20003, 'Account not found');
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END transfer_funds;
/

CREATE OR REPLACE PROCEDURE mini_statement (p_acc IN NUMBER) IS
  CURSOR c_txn IS
    SELECT txn_id, txn_type, amount, TO_CHAR(txn_time, 'DD-MON-YYYY') AS t
    FROM transactions
    WHERE acc_no = p_acc
    ORDER BY txn_id DESC;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Mini statement for account ' || p_acc);
  FOR r IN c_txn LOOP
    EXIT WHEN c_txn%ROWCOUNT > 5;
    DBMS_OUTPUT.PUT_LINE(r.txn_id || '  ' || RPAD(r.txn_type, 13) ||
                         TO_CHAR(r.amount, '999,990.00') || '  ' || r.t);
  END LOOP;
END mini_statement;
/
SHOW ERRORS
```

6 Schedules and Serializability

Let T_1 be a withdrawal from account A and T_2 another withdrawal from the same account A. Table 3 lists three schedules.

Table 3: Example schedules on data item A

Schedule	**Order of operations**	**Result**
S_1 (serial)	$R_1(A)\, W_1(A)\, R_2(A)\, W_2(A)$	Correct; serial.
S_2 (non-serial)	$R_1(A)\, W_1(A)\, R_2(A)\, W_2(A)\, R_1(B)$ with T_2 touching A only after T_1 finished with it	Conflict serializable, equivalent to $T_1 \to T_2$.
S_3 (non-serial)	$R_1(A)\, R_2(A)\, W_1(A)\, W_2(A)$	Not conflict serializable; lost update.

In S_3, $R_1(A)$ comes before $W_2(A)$, giving the edge $T_1 \to T_2$, and $R_2(A)$ comes before $W_1(A)$, giving $T_2 \to T_1$. The precedence graph in Figure 3 has a cycle, so S_3 is not conflict serializable. This is exactly the lost-update schedule reproduced in Test 5 below.

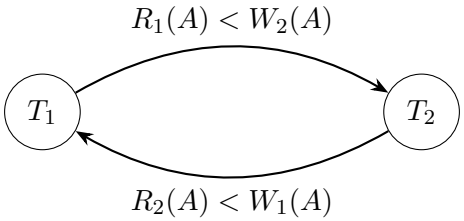


Figure 3: Precedence graph of S_3 (cycle, so not serializable)

Recoverability. Oracle never lets a session read another session's uncommitted data, so there are no dirty reads. A transaction can therefore never depend on one that later rolls back, which means schedules are recoverable and cascading rollbacks do not occur.

Lock-based protocol. The procedures follow strict two-phase locking: locks are acquired while the work proceeds (growing phase) and all are released together at `COMMIT` or `ROLLBACK` (shrinking phase). Strict 2PL guarantees conflict-serializable schedules.

7 Execution and Results

All tests were planned for Oracle 10g SQL*Plus. Start with:

```
C:\> sqlplus username/password
SQL*Plus: Release 10.2.0.1.0 - Production on Wed Oct 7 2026
SQL> SET SERVEROUTPUT ON
SQL> @sql/01_setup.sql
SQL> @sql/02_procedures.sql
```

Initial data (`SELECT acc_no, balance FROM accounts ORDER BY acc_no;`):

```
    ACC_NO     BALANCE
---------- -----------
       101   10,000.00
       102    5,000.00
       103   20,000.00
```

7.1 Test 1: Successful Transfer (Atomicity and Durability)

```
SQL> EXEC transfer_funds(101, 102, 2000);
PL/SQL procedure successfully completed.

SQL> SELECT acc_no, balance FROM accounts ORDER BY acc_no;

    ACC_NO     BALANCE
---------- -----------
       101    8,000.00
       102    7,000.00
       103   20,000.00

SQL> SELECT txn_id, acc_no, txn_type, amount, related_acc FROM transactions;

    TXN_ID     ACC_NO TXN_TYPE         AMOUNT RELATED_ACC
---------- ---------- ------------ ---------- -----------
      1001        101 TRANSFER_OUT   2,000.00         102
      1002        102 TRANSFER_IN    2,000.00         101

SQL> SELECT log_id, acc_no, old_balance, new_balance FROM audit_log;

    LOG_ID     ACC_NO OLD_BALANCE NEW_BALANCE
---------- ---------- ----------- -----------
         1        101   10,000.00    8,000.00
         2        102    5,000.00    7,000.00
```

Both balances changed together, two transaction records were written, and the trigger logged both updates.

7.2 Test 2: Deposit, Withdrawal and Mini Statement (Cursor)

```
SQL> EXEC deposit(103, 5000);
PL/SQL procedure successfully completed.

SQL> EXEC withdraw(101, 1500);
PL/SQL procedure successfully completed.

SQL> EXEC mini_statement(101);
Mini statement for account 101
1004  WITHDRAW       1,500.00  07-OCT-2026
1001  TRANSFER_OUT   2,000.00  07-OCT-2026
PL/SQL procedure successfully completed.

SQL> SELECT acc_no, balance FROM accounts ORDER BY acc_no;

    ACC_NO     BALANCE
---------- -----------
       101    6,500.00
       102    7,000.00
       103   25,000.00
```

7.3 Test 3: Failed Transfer Is Rolled Back (Atomicity)

Account 102 holds 7,000, so a transfer of 50,000 must fail and leave no trace.

```
SQL> EXEC transfer_funds(102, 103, 50000);
BEGIN transfer_funds(102, 103, 50000); END;
*
ERROR at line 1:
ORA-20001: Insufficient funds in account 102
(ORA-06512 stack lines omitted)

SQL> SELECT acc_no, balance FROM accounts ORDER BY acc_no;

    ACC_NO     BALANCE
---------- -----------
       101    6,500.00
       102    7,000.00
       103   25,000.00

SQL> SELECT COUNT(*) FROM transactions;

  COUNT(*)
----------
         4

SQL> SELECT COUNT(*) FROM audit_log;

  COUNT(*)
----------
         4
```

Balances, the transaction table and the audit table are unchanged, which shows atomicity.

7.4 Test 4: Integrity Constraint (Consistency)

```
SQL> UPDATE accounts SET balance = -100 WHERE acc_no = 103;
UPDATE accounts SET balance = -100 WHERE acc_no = 103
*
ERROR at line 1:
ORA-02290: check constraint (SCOTT.CHK_BALANCE) violated
```

(The schema name in the message is the user who runs the script.) The database refuses a state that breaks the rule, so consistency is preserved.

Before the concurrency tests, run `@sql/01_setup.sql` and `@sql/02_procedures.sql` again to restore the starting data (account 101 = 10,000; 102 = 5,000). Open two SQL*Plus windows and log in to the same user in both.

7.5 Test 5: Lost Update Without Locking (Isolation Problem)

The application reads the balance into a variable and writes back a computed value, a common read-then-write pattern. T_1 withdraws 500 and T_2 withdraws 1,000 from account 101, so the correct final balance is 8,500.

Table 4: Timeline of the lost update

Time	Session A	Session B
t_1	reads balance = 10,000	
t_2		reads balance = 10,000
t_3	writes 9,500 and commits	
t_4		writes 9,000 and commits

Session A:

```
SQL> VARIABLE bal NUMBER
SQL> EXEC SELECT balance INTO :bal FROM accounts WHERE acc_no = 101;
SQL> UPDATE accounts SET balance = :bal - 500 WHERE acc_no = 101;
1 row updated.
SQL> COMMIT;
Commit complete.
```

Session B (reads before A commits, writes after):

```
SQL> VARIABLE bal NUMBER
SQL> EXEC SELECT balance INTO :bal FROM accounts WHERE acc_no = 101;
SQL> UPDATE accounts SET balance = :bal - 1000 WHERE acc_no = 101;
1 row updated.
SQL> COMMIT;
Commit complete.

SQL> SELECT balance FROM accounts WHERE acc_no = 101;

    BALANCE
-----------
   9,000.00
```

The final balance is 9,000 instead of 8,500: Session A's withdrawal of 500 has been lost. This is schedule S_3 from Section 6.

7.6 Test 6: Lost Update Fixed With Row Locking

Restore the data and repeat, but read with `FOR UPDATE`. Session A locks the row first, so Session B must wait.

Session A:

```
SQL> EXEC SELECT balance INTO :bal FROM accounts WHERE acc_no = 101 FOR UPDATE;
```

Session B (the command waits, because A holds the lock):

```
SQL> EXEC SELECT balance INTO :bal FROM accounts WHERE acc_no = 101 FOR UPDATE;
(waiting for Session A ...)
```

Session A then finishes:

```
SQL> UPDATE accounts SET balance = :bal - 500 WHERE acc_no = 101;
1 row updated.
SQL> COMMIT;
Commit complete.
```

Only now does Session B continue; it reads the committed value 9,500 and finishes:

```
PL/SQL procedure successfully completed.
SQL> UPDATE accounts SET balance = :bal - 1000 WHERE acc_no = 101;
1 row updated.
SQL> COMMIT;
Commit complete.

SQL> SELECT balance FROM accounts WHERE acc_no = 101;

    BALANCE
-----------
   8,500.00
```

The correct balance of 8,500 is obtained, so the lock-based protocol removed the lost update.

7.7 Test 7: Deadlock (ORA-00060)

Restore the data again. Two transfers in opposite directions each lock their own account first and then try to lock the other account.

Session A:

```
SQL> UPDATE accounts SET balance = balance - 100 WHERE acc_no = 101;
1 row updated.
```

Session B:

```
SQL> UPDATE accounts SET balance = balance - 200 WHERE acc_no = 102;
1 row updated.
```

Table 5: Timeline of the deadlock

Time	**Session A (101 → 102)**	**Session B (102 → 101)**
t_1	updates account 101	
t_2		updates account 102
t_3	updates account 102 (waits for B)	
t_4		updates account 101 (waits for A: cycle)

Session A (waits):

```
SQL> UPDATE accounts SET balance = balance + 100 WHERE acc_no = 102;
(waiting for Session B ...)
```

Session B closes the cycle. After a few seconds Oracle's deadlock detector chooses a victim, normally the session that completed the cycle:

```
SQL> UPDATE accounts SET balance = balance + 200 WHERE acc_no = 101;
UPDATE accounts SET balance = balance + 200 WHERE acc_no = 101
*
ERROR at line 1:
ORA-00060: deadlock detected while waiting for resource
```

Only the failed statement is rolled back; Session B still holds its lock on account 102, so it must roll back to release it:

```
SQL> ROLLBACK;
Rollback complete.
```

Session A's waiting statement now completes and A commits:

```
1 row updated.
SQL> COMMIT;
Commit complete.

SQL> SELECT acc_no, balance FROM accounts WHERE acc_no IN (101,102) ORDER BY acc_no;

    ACC_NO     BALANCE
---------- -----------
       101    9,900.00
       102    5,100.00
```

The database detected the deadlock, aborted one transaction and let the other finish, but the aborted transfer had to be retried by the application.

7.8 Test 8: Deadlock Prevention by Ordered Locking

`transfer_funds` always locks the lower account number first, whichever direction the money moves. Now both sessions try to lock account 101 first. **Session A:**

```
SQL> SELECT balance FROM accounts WHERE acc_no = 101 FOR UPDATE;

    BALANCE
-----------
   9,900.00
```

Session B (waits at the very first lock, holding nothing, so no cycle can form):

```
SQL> SELECT balance FROM accounts WHERE acc_no = 101 FOR UPDATE;
(waiting for Session A ...)
```

Session A locks 102, moves 100 from 101 to 102 and commits; Session B then proceeds normally and moves 200 from 102 to 101. No ORA-00060 appears, and the final balances are 10,000 and 5,000 again, as they should be:

```
    ACC_NO     BALANCE
---------- -----------
       101   10,000.00
       102    5,000.00
```

Because every transaction requests locks in the same global order, a circular wait is impossible, so deadlock is prevented rather than merely detected.

7.9 Test 9: Lock Wait Control (NOWAIT) and Serializable Isolation

With Session A holding the lock on account 101, Session B can refuse to wait:

```
SQL> SELECT balance FROM accounts WHERE acc_no = 101 FOR UPDATE NOWAIT;
SELECT balance FROM accounts WHERE acc_no = 101 FOR UPDATE NOWAIT
*
ERROR at line 1:
ORA-00054: resource busy and acquire with NOWAIT specified
```

In serializable mode, Session B takes a snapshot first. If Session A changes and commits the same row afterwards, B's later update is refused:

```
SQL> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Transaction set.
SQL> SELECT balance FROM accounts WHERE acc_no = 101;
(Session A now updates account 101 and commits)
SQL> UPDATE accounts SET balance = balance - 1000 WHERE acc_no = 101;
UPDATE accounts SET balance = balance - 1000 WHERE acc_no = 101
*
ERROR at line 1:
ORA-08177: can't serialize access for this transaction
```

Table 6: Summary of experiments

Test	**Scenario**	**Outcome**
1	Transfer 101 → 102	Debit, credit, records and audit saved together.
2	Deposit, withdrawal, cursor statement	Correct balances; statement lists latest entries.
3	Transfer with insufficient funds	Rolled back; nothing changed.
4	Negative balance	Rejected by CHECK constraint (ORA-02290).
5	Concurrent read-then-write	Lost update (9,000 instead of 8,500).
6	Same test with FOR UPDATE	Correct result (8,500).
7	Opposite transfers, no ordering	Deadlock detected (ORA-00060).
8	Ordered locking	No deadlock; correct balances.
9	NOWAIT and serializable mode	ORA-00054 and ORA-08177 as expected.

8 Results and Discussion

The tests show that the stored procedures keep the bank data correct. Failed operations leave no trace, constraints stop invalid values, and the audit table records every balance change automatically. The concurrency experiments show that default behaviour is not enough: a read-then-write pattern loses updates, and two transfers moving in opposite directions can deadlock. Row locking with `FOR UPDATE` fixes the first problem, and acquiring locks in a fixed order fixes the second.

The trade-off is concurrency: sessions that touch the same account must wait for each other, so locks should be held for as short a time as possible. Because Oracle uses row-level locks, transfers between unrelated accounts still run in parallel.

9 Mapping POs and SDGs

9.1 Program Outcomes (POs) Mapping

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied database, transaction and concurrency-control principles.
PO2	Problem Analysis	Identified key updates and deadlocks in concurrent banking operations.
PO3	Design/Development of Solutions	Designed the schema, procedures and locking strategy.
PO5	Modern Tool Usage	Used Oracle 10g, SQL*Plus and PL/SQL.
PO6	Engineer and Society	Protects customers' money through reliable, auditable processing.

Table 7: Mapping of Program Outcomes

9.2 Sustainable Development Goals (SDGs) Mapping

SDG No.	Goal	Relevance
SDG 8	Decent Work and Economic Growth	Reliable financial transactions support trusted banking services.
SDG 9	Industry, Innovation, and Infrastructure	Demonstrates dependable digital infrastructure for financial systems.
SDG 16	Peace, Justice, and Strong Institutions	The audit trail promotes transparency and accountability.

Table 8: Mapping of Sustainable Development Goals

10 Conclusion

SafeTransfer shows how a database management system keeps money transfers correct, both when a failure occurs and when many users work at the same time. Stored procedures that commit when every step succeeds and roll back on any error give atomicity and durability. Integrity constraints keep the data consistent by rejecting invalid values, such as a negative balance. The audit trigger records every change in a balance, so each transaction can be traced.

The concurrency experiments showed why these features alone are not enough. A read-then-write pattern produced a lost update, with a final balance of 9,000 instead of 8,500. Two opposite transfers produced a deadlock, which Oracle detected with error `ORA-00060` after one transfer had already been lost. Row locking with `SELECT ... FOR UPDATE` gave the correct result of 8,500, and locking the lower account number first removed the circular wait, so no deadlock occurred. Preventing a deadlock therefore proved better than detecting it and retrying.

The project also connected these results to the theory taught in the course. The lost-update schedule gives a cycle in the precedence graph, so it is not conflict serializable, and strict two-phase locking gives schedules that are serializable and recoverable. In this way, the project turns the ACID properties and lock-based concurrency control of Units IV and V into repeatable, working examples.

11 Future Scope

- **Automatic retry after deadlock:** The application could catch `ORA-00060` and retry the failed transfer by itself, so the user never sees the error.
- **Multiple types of transfer:** Features such as standing instructions, transfers to other banks, and daily transfer limits could be added.
- **Views and reports:** Views joining customers and accounts, and reports using `GROUP BY` and `HAVING`, would give managers balance sheets and totals per branch or city.
- **Timestamp-based concurrency control:** The same experiments could be repeated with timestamp ordering and compared with locking on performance and waiting time.
- **Performance tests with more users:** A script could run hundreds of concurrent transfers to measure throughput, lock waiting, and deadlock frequency, and to study how indexes affect them.
- **Crash recovery:** A crash could be simulated during a transfer, and the redo and undo logs could be used to study recovery in the way ARIES describes.

Bibliography

[**1**] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.

[**2**] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill, 2003.

[**3**] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, 1993.

[**4**] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, 1983.

[**5**] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Communications of the ACM*, vol. 19, no. 11, pp. 624–633, 1976.

[**6**] Oracle Corporation, *Oracle Database Concepts, 10g Release 2 (10.2)*, Data Concurrency and Consistency chapter.